

Verdant-Search: Bridging Vision and Text in Distributed Search Through Multimodal Learning-to-Rank

Student: CAI HONGYI, S2175463

Supervisor: CHIAM YIN KIA

University of Malaya

January 24, 2026

Overview

1. Introduction
2. Problem Statement and Objectives
3. Literature Review
4. Methodology
5. Requirements Specification
6. System Analysis and Design
7. Technical Implementation

Introduction: The Multimodal Search Gap

Enterprise Documents Are Multimodal

- Business reports, technical docs contain critical visual information
- Charts, tables, diagrams convey key insights beyond text

Current Search Systems Are Text Only

- Commercial solutions ignore visual text mixed content
- Reranking methods (RankZephyr, classical LTR) operate purely on text

RAG Systems Lack Iterative Refinement

- Limited mechanism for LLM insights to improve retrieval
- Static search without conversational optimization

Core Needs

- Fast, scalable search over million document collections
- Rank documents considering text and visual quality
- Conversational AI that actively improves search

Problem Statement

ID	Problem Description
P1	Scalability Challenges in Multimodal Search. Multimodal search requires expensive vision language inference and coordinated indexing. Without proper optimization, systems struggle to achieve subsecond latency at enterprise scale.
P2	Absence of Multimodal Reranking. All existing learning to rank methods process only text. Current approaches lack listwise reasoning over cross modal evidence, leaving visual content unranked.
P3	Limited Enterprise Knowledge Access. Organizations lack bidirectional integration where search improves AI responses and AI insights refine search. Current RAG systems treat retrieval as one shot preprocessing.

Research Objectives

ID	Objective Description
O1	To identify existing search engine architectures, learning to rank approaches, vision language models, and retrieval augmented generation techniques through literature review.
O2	To design and develop a multimodal search engine integrating distributed crawling, hybrid retrieval, multimodal listwise reranking, and iterative RAG with query rewriting.
O3	To evaluate system effectiveness through ranking metrics, latency benchmarks, and answer quality assessments.

Literature Review: Search Engine Architecture (1/9)

Approach	Key Contributions	Limitations
PageRank [1]	Link based quality indicators; revolutionized web search ranking	Requires link structure; susceptible to manipulation
Tripartite Architecture [2]	Crawlers, indexers, query processors framework	Text only; single modality
Inverted Index [3]	Efficient keyword search at web scale; posting lists	Lexical matching only
Neural IR [4]	Semantic understanding beyond keyword matching	Computational overhead
Dense Passage Retrieval [5]	BERT based semantic search; zero lexical overlap retrieval	Training data requirements

Selection Rationale

Selected: Hybrid architecture combining inverted index (BM25) with dense retrieval (DPR).

Reason: Combines efficiency of keyword search with semantic understanding. Pure keyword methods miss semantic matches; pure neural methods have high latency.

Literature Review: Keyword Based Search Methods (2/9)

Method	Mechanism	Strength	Weakness
TF IDF [6]	Term frequency inverse document frequency weighting	Simple, interpretable	Length bias
BM25 [7]	Probabilistic with saturation and length normalization	Robust baseline; tunable parameters	Lexical gap
Contriever [8]	Unsupervised contrastive dense retrieval	No labeled data needed	Less effective than supervised

Selection Rationale

Selected: BM25 for first stage keyword retrieval.

Reason: BM25 remains competitive with neural methods on many benchmarks while offering millisecond latency. TF IDF lacks saturation handling; Contriever requires GPU inference for first stage.

Literature Review: Vector Embeddings (3/9)

Model	Architecture	Strength	Weakness
Word2Vec [9]	Skip gram, CBOW	Fast training; captures analogies	Context independent
GloVe [10]	Global matrix factorization	Leverages corpus statistics	Static embeddings
BERT [11]	Bidirectional transformer	Deep contextualization	Expensive inference
Sentence BERT [12]	Siamese BERT	Efficient similarity	Fixed representation
ColBERT [13]	Late interaction	Fine grained matching	Storage overhead

Selection Rationale

Selected: Sentence BERT for text embeddings; CLIP for multimodal alignment.

Reason: Sentence BERT balances quality and efficiency for text. ColBERT has high storage costs. Word2Vec and GloVe lack contextualization needed for semantic search.

Literature Review: Approximate Nearest Neighbor (4/9)

Algorithm	Index Structure	Query Complexity	Trade off
Brute Force KNN	None	$O(N)$	Exact but slow
Product Quantization [14]	Learned codebooks	$O(N)$ with fast distance	Memory vs accuracy
IVF	Inverted file clusters	$O(N/k)$	Partition quality critical
HNSW [15]	Hierarchical proximity graphs	$O(\log N)$	Build time vs query speed
ANCE [16]	Hard negative mining	Training time	Improved recall

Selection Rationale

Selected: HNSW for vector search index.

Reason: HNSW achieves logarithmic query complexity with high recall. IVF requires careful cluster tuning; PQ trades too much accuracy for memory. HNSW is the industry standard for production vector search.

Literature Review: Classical Learning to Rank (5/9)

Method	Paradigm	Loss Function	Strength	Weakness
Regression	Pointwise	MSE	Simple	Ignores ordering
RankNet [17]	Pairwise	Pairwise cross entropy	Relative preferences	Quadratic pairs
LambdaRank [18]	Pairwise	NDCG gradient	Direct metric optimization	Complex gradients
LambdaMART [19]	Pairwise	Boosted trees	Robust; interpretable	Feature engineering
ListNet [20]	Listwise	Top one probability	Full list context	Permutation cost
ListMLE [21]	Listwise	MLE over permutations	Principled	Computational

Selection Rationale

Selected: Listwise paradigm for reranking (extended to LLM based).

Reason: Listwise methods consider full ranking context. LambdaMART requires extensive feature engineering; pointwise ignores document relationships. Listwise enables direct metric optimization.

Literature Review: LLM Based Ranking (6/9)

Method	Approach	Strength	Weakness
monoBERT [22]	Cross encoder scoring	High accuracy	$O(N)$ inference
RankGPT [23]	GPT 4 permutation generation	Zero shot; SOTA	High API cost; latency
RankZephyr [24]	Distilled setwise prompting	Efficient; open source	Requires fine tuning
Pairwise LLM [25]	LLM pairwise comparison	Effective	Quadratic comparisons

Selection Rationale

Selected: RankZephyr style setwise prompting extended to multimodal.

Reason: RankZephyr balances effectiveness with efficiency through setwise comparison. RankGPT is too expensive for production; pairwise methods have quadratic complexity. Setwise enables listwise ranking with bounded comparisons.

Literature Review: Vision Language Models (7/9)

Model	Training	Scale	Strength	Weakness
CLIP [26]	Contrastive	400M pairs	Zero shot; efficient	Short text focus
ALIGN [27]	Contrastive	1.8B noisy	Scale robust	Noise sensitivity
ALBEF [28]	Momentum distillation	14M pairs	Alignment quality	Moderate scale
BLIP [29]	Bootstrapping	129M pairs	Generation capable	Training complexity
BLIP 2 [30]	Frozen encoders + Q Former	129M pairs	LLM integration	Inference cost

Selection Rationale

Selected: CLIP for image text alignment; BLIP 2 for multimodal reasoning in reranker.

Reason: CLIP provides efficient zero shot image text matching for first stage retrieval. BLIP 2 enables sophisticated cross modal reasoning needed for listwise reranking by connecting to LLMs.

Literature Review: RAG Systems (8/9)

Method	Mechanism	Strength	Weakness
RAG [31]	Retrieve then generate	Grounds LLM outputs	One shot retrieval
REALM [32]	Joint training	End to end optimization	Training complexity
Self RAG [33]	Self reflection tokens	Adaptive retrieval	Additional training
CRAG [34]	Corrective strategies	Relevance evaluation	External web search
Conversational Search [35]	Dialog context	Natural interaction	History management

Selection Rationale

Selected: Iterative RAG with query rewriting inspired by Self RAG and conversational search.

Reason: Standard RAG lacks refinement capability. Self RAG and CRAG treat retrievers as black boxes. Our approach closes the loop where LLM insights improve subsequent retrieval through query rewriting.

Literature Review: Benchmark Datasets (9/9)

Dataset	Queries	Documents	Domain	Modality
MS MARCO [36]	1M train	8.8M	Web	Text only
TREC DL [37]	200 600/yr	From MARCO MS	Web	Text only
BEIR [38]	Varies	18 datasets	Multi domain	Text only
MMDocIR [39]	1.7K expert	313 docs	10 domains	Text + Image

Selection Rationale

Selected: MS MARCO and TREC DL for text ranking; MMDocIR for multimodal evaluation.

Reason: MS MARCO provides large scale training signal. TREC DL offers dense judgments for reliable evaluation. MMDocIR is the only benchmark with multimodal document retrieval tasks, essential for validating our multimodal reranker.

Research Gap Summary

Gap	Description	Our Contribution
Gap 1	All existing LTR methods operate exclusively on text; reranking stage unexplored for multimodal content	First multimodal listwise reranker extending setwise prompting to vision language models
Gap 2	RAG systems assume text only retrieval; visual evidence underutilized	Multimodal RAG integrating image and text retrieval with generation
Gap 3	Current systems treat search as one shot; limited iterative refinement	Closed loop search RAG cycle where LLM insights improve subsequent retrieval
Gap 4	Vision language model inference expensive; efficiency challenges at scale	Two stage ranking with efficient first stage and bounded multimodal reranking

Existing System Comparison

Capability	Ours	Google	Perplexity	MMDocIR	RankZephyr	Traditional
Hybrid Retrieval	✓	✓	✓			
Multimodal Retrieval	✓	✓		✓		
Multimodal LTR Reranker	✓	?				
Listwise Ranking	✓	?			✓	
LLM Integration	✓	✓	✓			
RAG Summarization	✓	✓	✓			
Iterative Search Feedback	✓					

Key Insight

Only system combining multimodal retrieval, multimodal reranking, and iterative LLM driven refinement in complete end to end architecture.

Methodology: Agile Development Approach

This project employed an **Agile development methodology** to handle evolving requirements through flexibility, collaboration, and iterative progress. Unlike traditional linear models, Agile embraces change via a continuous feedback loop with stakeholders. To provide structure, the project was managed using **Scrum**, a popular and effective Agile framework.

Scrum Roles

- **Product Owner:** Key stakeholders at Infinity Datatech, primarily the Product Manager and technical leads, who defined the project vision and prioritized features based on organizational knowledge management needs.
- **Development Team:** A self contained role filled by the developer on this academic project, responsible for building the full stack search engine system.
- **Scrum Master:** A dual role fulfilled by the developer (process facilitation) and the academic supervisor (guidance and impediment removal).

Scrum Artifacts

- **Product Backlog:** A master list of all features and requirements, prioritized in collaboration with Infinity Datatech based on technical value and enterprise search needs.
- **Sprint Backlog:** A subset of high priority tasks selected from the Product Backlog to be completed within a two week Sprint.
- **Increment:** The tangible, working, and tested piece of software produced at the end of each Sprint, adding to previous functionality with deployable search components.

Scrum Events

The work was structured around recurring events within each two week Sprint:

1. **Sprint Planning:** Selecting items from the Product Backlog to define the Sprint Goal for search engine components.
2. **Daily Check-ins:** Brief, regular syncs to discuss progress and obstacles in development.
3. **Sprint Review:** A demonstration of the Sprint increment to Infinity Datatech stakeholders to gather critical feedback on search quality and system performance.
4. **Sprint Retrospective:** An internal reflection to identify process improvements for the next Sprint.

The Scrum Framework - Process Flow

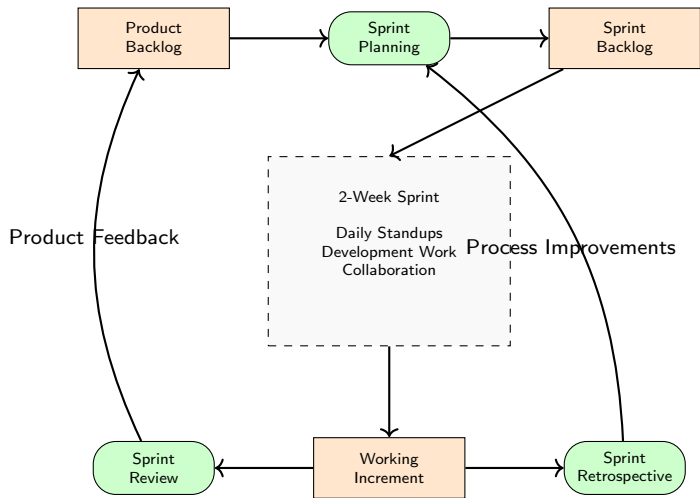


Figure: The Scrum Framework - Process Flow

Project Management and Workflow Tools

To support this Agile Scrum process, a specific set of digital tools was employed:

- **GitHub:** Used for version control and managing the **Product** and **Sprint Backlogs** via Projects and Issues features for tracking search engine development.
- **Microsoft Teams:** Used for formal academic supervision meetings.
- **Lark:** Serves as the central collaboration hub for daily communication, task tracking, and sprint management, coordinating distributed system components and data ingestion pipelines.

Product Manager Interview - Requirements Discovery

Example Q: Can you describe the current challenges your technical teams face when searching for internal documentation?

Outcome:

- Engineers spend 30-45 minutes daily searching across fragmented systems (Confluence, Google Drive, Git repositories, local shares).
- Existing keyword search misses conceptually related documents due to terminology variations.
- Critical visual information in charts and diagrams is not searchable, forcing manual document browsing.
- No unified interface, requiring context switching between multiple tools.

Example Q: What types of queries do your teams typically need answers for?

Outcome:

- Technical how-to questions requiring code examples and architecture diagrams.
- Troubleshooting queries needing logs, error messages, and past incident reports.
- Policy and compliance questions requiring exact citations from official documents.
- Comparative analysis requiring synthesis across multiple technical specifications.
- This revealed need for both precise retrieval and intelligent summarization.

Product Manager Interview - Feature Priorities

Example Q: If you had to prioritize three capabilities for the search system, what would they be?

Outcome:

- **Priority 1:** Multimodal search that can find documents based on visual content descriptions, not just text.
- **Priority 2:** Conversational interface where follow up questions refine results without starting over.
- **Priority 3:** Source attribution with exact citations so answers can be verified and trusted.
- Additional need: Sub-second query response time for daily usage adoption.

Example Q: How do you currently evaluate whether search results are relevant?

Outcome:

- Manual inspection of top 10 results for precision.
- User feedback on whether they found what they needed.
- Time to complete search task as efficiency metric.
- This informed our evaluation methodology using NDCG@10, MRR, and latency measurements.

Workflow Analysis

Detailed observations of Infinity Datatech manual search processes were conducted:

- **Process mapping:** Documented the end to end workflow from query formulation to answer discovery, revealing an average of 7 context switches per search session across different data sources.
- **Time motion studies:** Measured time spent on manual tasks to identify bottlenecks. Average search session: 12 minutes, with 60 percent spent navigating between systems rather than evaluating content.
- **Artifact analysis:** Examined search logs and user feedback to understand information needs. Found 40 percent of queries require visual evidence and 65 percent benefit from conversational refinement.
- **Tool evaluation:** Assessed the current toolset (Confluence search, Google Drive search, file system grep) to find gaps. No system provided multimodal retrieval or intelligent ranking beyond keyword matching.

Hierarchical Task Analysis

The Hierarchical Task Analysis below outlines the system functional structure, decomposed into eight primary modules. This task breakdown forms the basis for the use cases and activity diagrams that follow.

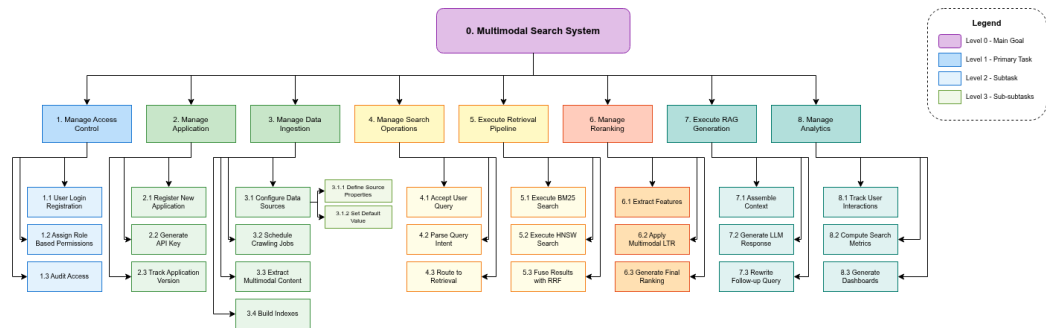


Figure: Hierarchical Task Analysis of Verdant Search System

Stakeholder Introduction

Infinity Data Tech Sdn. Bhd.

- Malaysian company specializing in VPN and AI customer service products
- Own application products and in house development
- Focus on software development and network protocol research

Project Context

- Internal knowledge base search for technical teams
- Customer support documentation retrieval
- VPN technical knowledge and censorship circumvention expertise

System Architecture Overview

Backend Pipeline

- Distributed web crawling with text and image extraction
- Dual indexing: Inverted index (BM25) + HNSW (CLIP embeddings)
- Compressed storage with incremental updates
- Two stage ranking: Fast retrieval + Multimodal LTR reranking

Frontend Interface

- Dual panel UI: Document list + AI chat
- Conversational query refinement
- Real time result updates
- Closed loop: User follow up leads to query rewriting leads to new retrieval leads to updated AI response

Project Modules Overview

Module Name	Description
Module 1: User Management	User authentication, authorization, and profile management with SSO integration.
Module 2: Search and Retrieval	Hybrid search (BM25 + HNSW vector) with multimodal reranking.
Module 3: Data Ingestion	Distributed crawling, document processing, chunking, and index building.
Module 4: Analytics and Monitoring	System metrics, search analytics, and performance monitoring.
Module 5: RAG Generation	LLM-based answer generation with context injection and citations.

Functional Requirements: Module 1 User Management

UC ID	FR ID	Functional Requirements	Priority
UC-01	FR-1.1	User shall register an account with email and password.	High
	FR-1.2	System shall validate email format and password strength.	High
	FR-1.3	System shall send verification email upon registration.	Medium
UC-02, UC-03	FR-1.4	User shall login with email/password or SSO authentication.	High
	FR-1.5	System shall support SSO providers (LDAP, Okta, Azure AD).	High
UC-07	FR-1.6	System shall authenticate user credentials securely.	High
UC-04	FR-1.7	User shall manage profile with configurable preferences.	Medium
	FR-1.8	User shall update display name, avatar, and notification settings.	Medium
UC-08	FR-1.9	User shall configure language and search settings.	Medium
UC-05	FR-1.10	User shall view and manage their chat history.	Medium
	FR-1.11	System shall display chat history with timestamps and summaries.	Medium
	FR-1.12	User shall export chat history in JSON or markdown format.	Low
UC-09	FR-1.13	System shall provide search functionality within chat history.	Medium
UC-11	FR-1.14	User shall be able to delete individual or bulk chat entries.	Medium
UC-06	FR-1.15	System shall implement session management with timeout.	Medium
	FR-1.16	System shall automatically logout after 24 hours of inactivity.	Medium
UC-10	FR-1.17	System shall validate session tokens on each request.	Medium

Functional Requirements: Module 2 Search and Retrieval

(1/2)

UC ID	FR ID	Functional Requirements	Priority
UC-12	FR-2.1	User shall submit search query through text input.	High
	FR-2.2	System shall sanitize and normalize query text.	High
	FR-2.3	System shall support query expansion with synonyms.	Medium
UC-16	FR-2.4	System shall perform keyword search using BM25.	High
	FR-2.5	System shall build inverted index with term posting lists.	High
UC-17	FR-2.6	System shall perform vector search using HNSW.	High
	FR-2.7	System shall encode queries with Sentence-BERT embeddings.	High
UC-18	FR-2.8	System shall support hybrid search with RRF fusion.	High
	FR-2.9	System shall combine BM25 and vector scores with weighted fusion.	High
UC-22	FR-2.10	System shall support pre-filtering on metadata fields.	Medium

Functional Requirements: Module 2 Search and Retrieval (2/2)

UC ID	FR ID	Functional Requirements	Priority
UC-13	FR-2.11 FR-2.12	System shall display search results as ranked document list. System shall show document snippets with highlighted keywords.	High High
UC-19	FR-2.13	System shall return ranked list of context chunks.	High
UC-20	FR-2.14 FR-2.15	System shall apply multimodal reranking to top candidates. System shall use BLIP-2 for cross-modal relevance scoring.	High High
UC-21	FR-2.16 FR-2.17	System shall extract and attach metadata to chunks. Metadata shall include filename, page number, and timestamp.	Medium Medium
UC-14	FR-2.18 FR-2.19	User shall ask follow-up questions to refine results. System shall track conversation history for context.	High High
UC-23	FR-2.20	System shall rewrite follow-ups into standalone queries.	High
UC-24	FR-2.21	System shall maintain conversation context across turns.	High
UC-15	FR-2.22	User shall filter results by metadata fields.	Medium

Functional Requirements: Module 3 Data Ingestion (1/2)

UC ID	FR ID	Functional Requirements	Priority
UC-25	FR-3.1 FR-3.2	Administrator shall configure data ingestion sources. System shall validate source credentials and permissions.	High High
UC-26	FR-3.3 FR-3.4 FR-3.5	System shall trigger distributed crawling jobs. System shall use Kafka for job coordination and task distribution. System shall implement exponential backoff for failed crawls.	High High Medium
UC-29	FR-3.6 FR-3.7	System shall crawl Confluence spaces via REST API. System shall respect Confluence rate limits and pagination.	High High
UC-30	FR-3.8 FR-3.9	System shall crawl Google Drive files via Drive API. System shall handle Google Docs, Sheets, Slides formats.	High High
UC-31	FR-3.10	System shall crawl Git repositories via Git protocol.	High
UC-32	FR-3.11	System shall crawl local network shares via SMB/NFS.	High
UC-33	FR-3.12 FR-3.13 FR-3.14	System shall extract text and images from documents. System shall use OCR for scanned PDF content. System shall extract images with captions from documents.	High Medium High

Functional Requirements: Module 3 Data Ingestion (2/2)

UC ID	FR ID	Functional Requirements	Priority
UC-34	FR-3.15	System shall chunk documents into semantic segments.	High
	FR-3.16	System shall maintain chunk overlap for context preservation.	High
UC-38	FR-3.17	System shall support chunking by heading structure.	Medium
UC-39	FR-3.18	System shall support chunking by paragraph boundaries.	Medium
UC-40	FR-3.19	System shall support fixed token count chunking.	Medium
UC-35	FR-3.20	System shall build inverted index for BM25.	High
	FR-3.21	System shall use Pyserini for efficient index construction.	High
UC-36	FR-3.22	System shall build HNSW vector index.	High
	FR-3.23	System shall use FAISS for scalable vector indexing.	High
UC-37	FR-3.24	System shall generate dense vector embeddings.	High
	FR-3.25	System shall use CLIP for image embeddings.	High
UC-27	FR-3.26	Administrator shall monitor indexing progress.	Medium
UC-28	FR-3.27	Administrator shall manage index refresh schedule.	Medium
	FR-3.28	System shall support cron-based scheduling.	Medium
UC-41	FR-3.29	System shall support real-time sync with webhooks.	Medium
UC-42	FR-3.30	System shall support scheduled batch updates.	High

Functional Requirements: Module 5 RAG Generation (1/2)

UC ID	FR ID	Functional Requirements	Priority
UC-43	FR-5.1 FR-5.2	System shall receive and process user query. System shall validate query length and format.	High High
UC-47	FR-5.3 FR-5.4	System shall inject retrieved context into LLM prompt. System shall format context with document metadata.	High High
UC-44	FR-5.5 FR-5.6	User shall view AI generated answer based on context. System shall display answer with markdown formatting.	High High
UC-51	FR-5.7 FR-5.8	LLM shall generate answers grounded in context. LLM shall avoid hallucination by using only retrieved content.	High High
UC-52	FR-5.9 FR-5.10	System shall add inline citations to answers. Citations shall link to source document and page number.	High High
UC-53	FR-5.11	System shall validate answer grounding with entailment check.	Medium
UC-54	FR-5.12	System shall indicate knowledge base limitations clearly.	High
UC-48	FR-5.13 FR-5.14	System shall stream LLM response in real-time. System shall use server-sent events for streaming.	High High

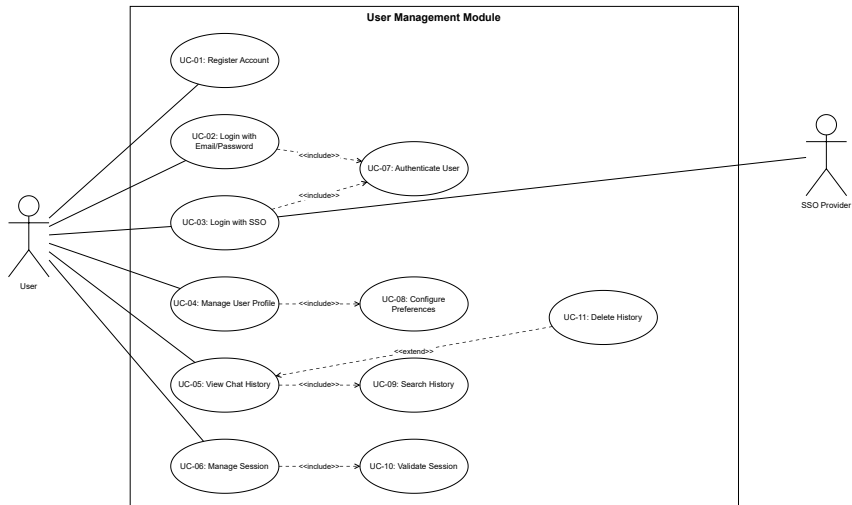
Functional Requirements: Module 5 RAG Generation (2/2)

UC ID	FR ID	Functional Requirements	Priority
UC-45	FR-5.15 FR-5.16	User shall ask follow-up questions. System shall detect follow-up vs new query intent.	High High
UC-49	FR-5.17 FR-5.18	System shall maintain conversation history. System shall limit history to last 10 conversation turns.	High Medium
UC-55	FR-5.19 FR-5.20	System shall rewrite follow-ups with context. System shall use LLM for query rewriting with history.	High High
UC-56	FR-5.21	System shall trigger new retrieval after query rewriting.	High
UC-59	FR-5.22	System shall update document list with new results.	High
UC-57	FR-5.23	System shall analyze user feedback on answers.	Medium
UC-58	FR-5.24	System shall refine search strategy based on feedback.	Medium
UC-46	FR-5.25 FR-5.26	User shall view source citations. System shall highlight cited text in source documents.	High Medium
UC-50	FR-5.27 FR-5.28	System shall format source references. References shall include title, author, and timestamp.	Medium Medium

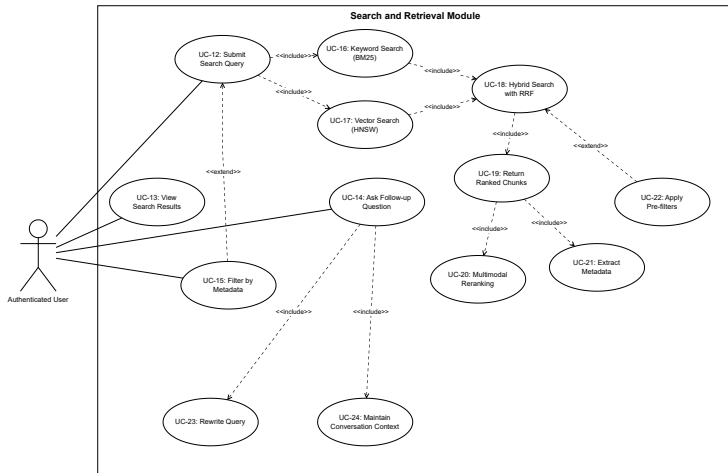
Non Functional Requirements

Req ID	Quality Attribute	Requirement Description	How to Achieve	How to Verify
NFR 1.1	Security: Authentication	System shall integrate with internal identity providers.	LDAP/SAML/OAuth2 with JWT	SSO login testing
NFR 1.2	Security: Confidentiality	Results shall adhere to file permissions; access validated.	ACL enforcement at retrieval	Access audit logs
NFR 1.3	Security: Integrity	All data in private infrastructure; LLM self-hosted.	On-premise + AES-256 encryption	Security audit
NFR 2.1	Performance: Time	P90 latency under 4 seconds from query to response.	Two-stage ranking + Redis cache	Prometheus monitoring
NFR 2.2	Performance: Capacity	System scales to 10 million document index.	Distributed HNSW + Kafka sharding	Load test with 10M docs
NFR 3.1	Reliability: Fault Tolerance	System implements error handling and logging.	Circuit breakers + retry + ELK logs	Chaos engineering tests
NFR 4.1	Maintainability: Modularity	Components independently deployable.	Microservices + Docker + K8s	Deployment isolation tests
NFR 5.1	Usability: Learnability	New users complete basic search within 5 minutes.	Intuitive UI + help tooltips	User testing (n=10+)

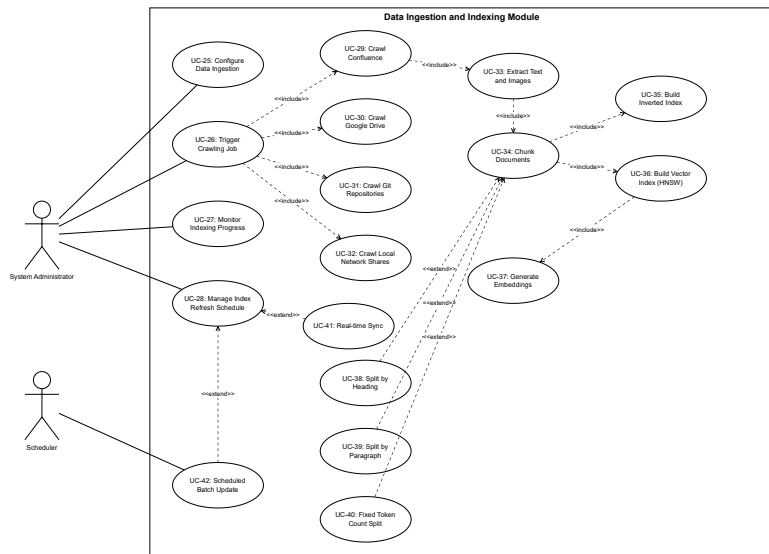
System Analysis and Design - Use Case Diagram



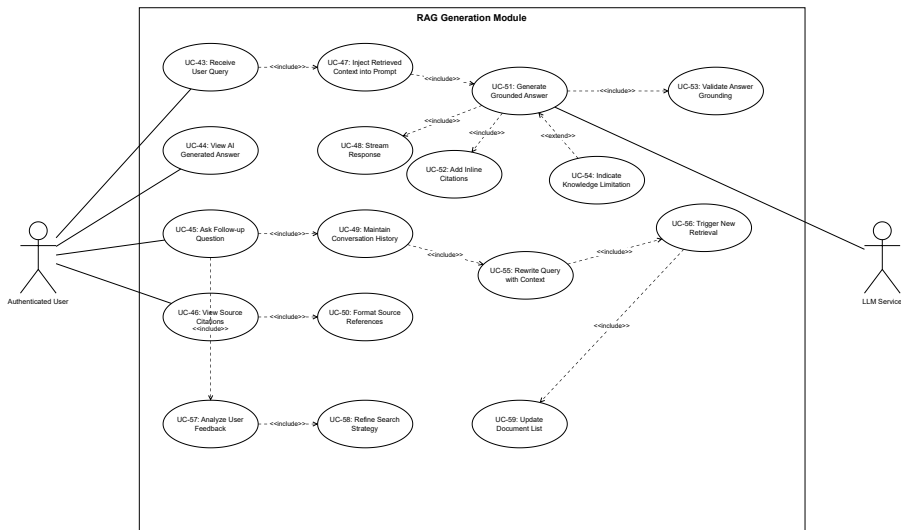
System Analysis and Design - Use Case Diagram



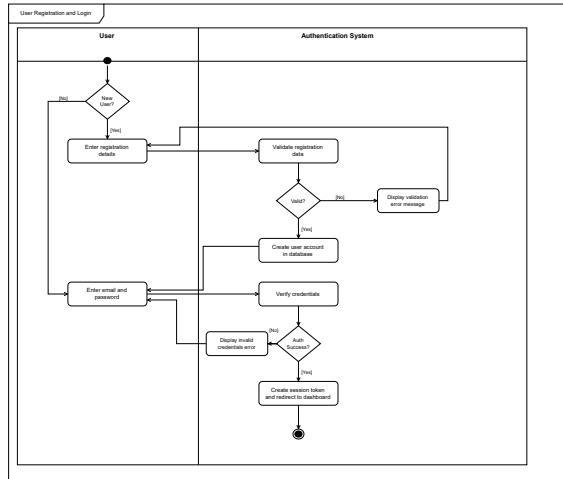
System Analysis and Design - Use Case Diagram



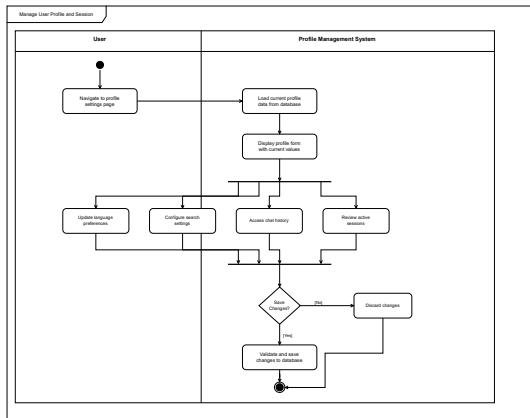
System Analysis and Design - Use Case Diagram



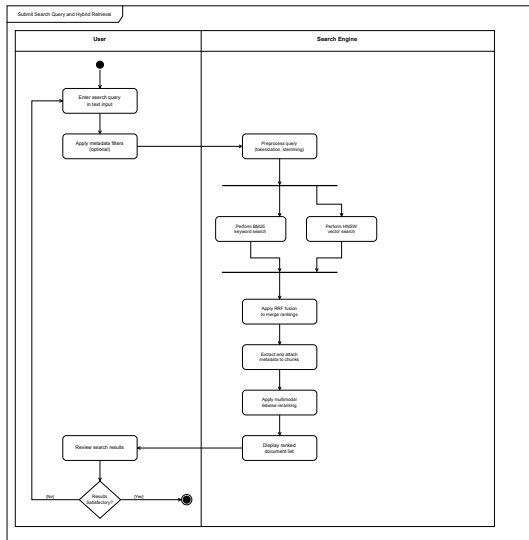
System Analysis and Design - Activity Diagram



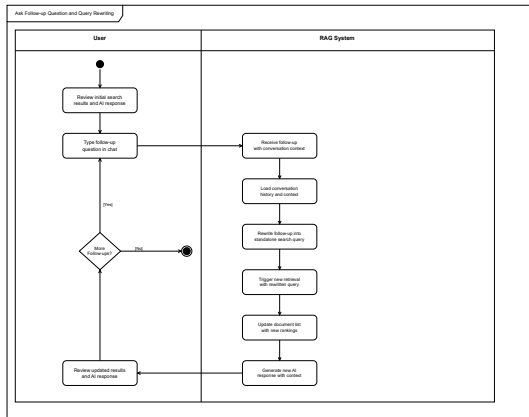
System Analysis and Design - Activity Diagram



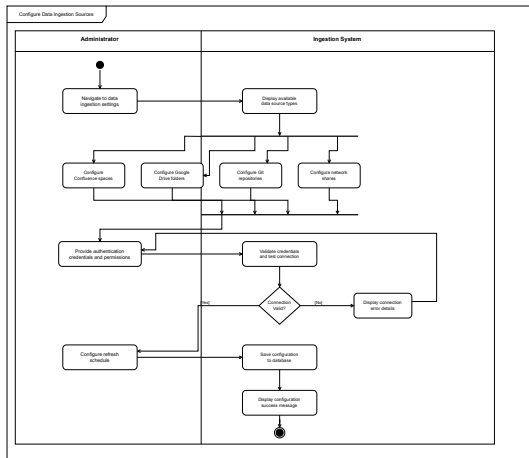
System Analysis and Design - Activity Diagram



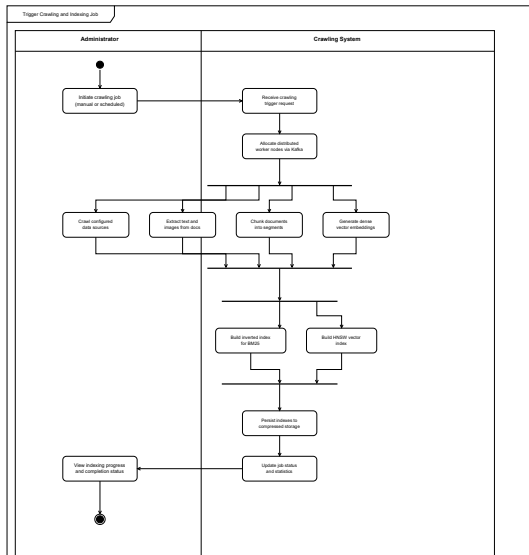
System Analysis and Design - Activity Diagram



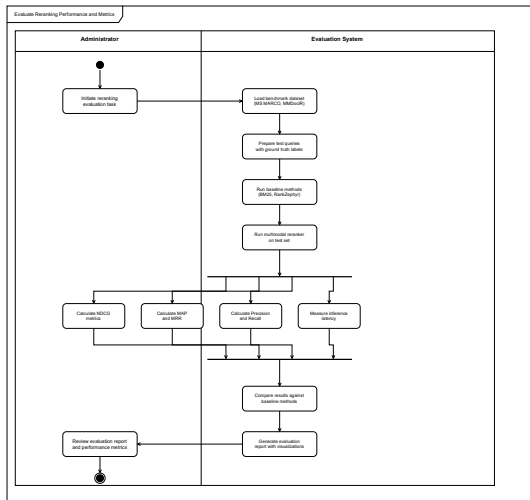
System Analysis and Design - Activity Diagram



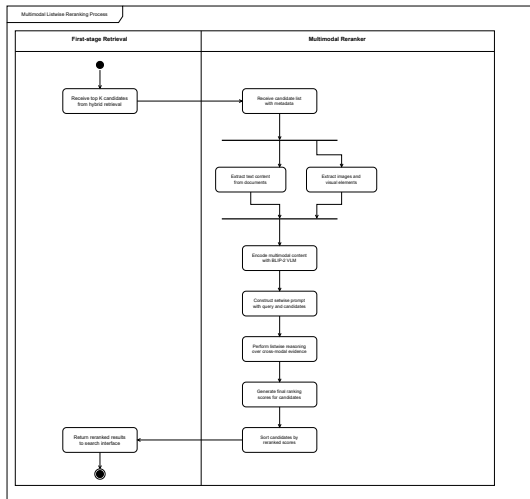
System Analysis and Design - Activity Diagram



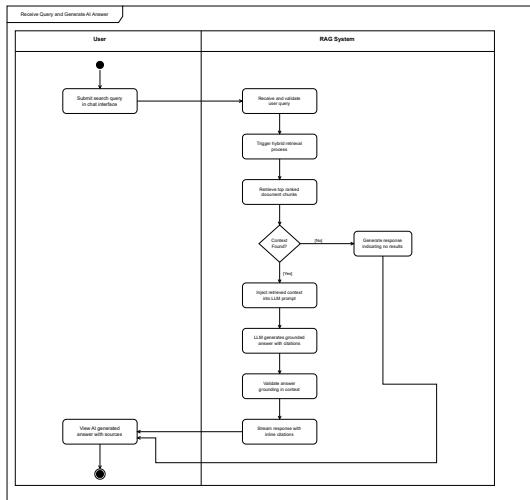
System Analysis and Design - Activity Diagram



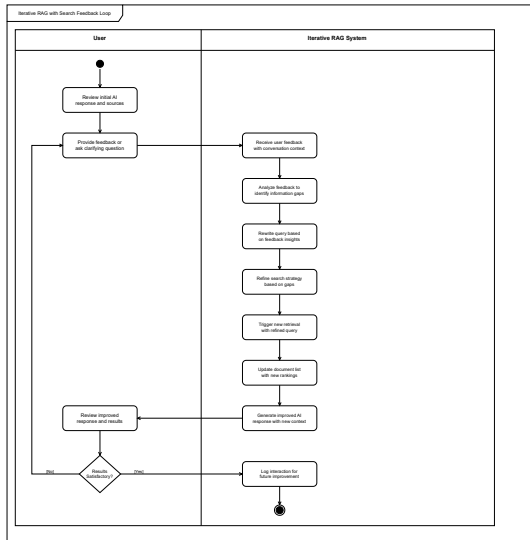
System Analysis and Design - Activity Diagram



System Analysis and Design - Activity Diagram

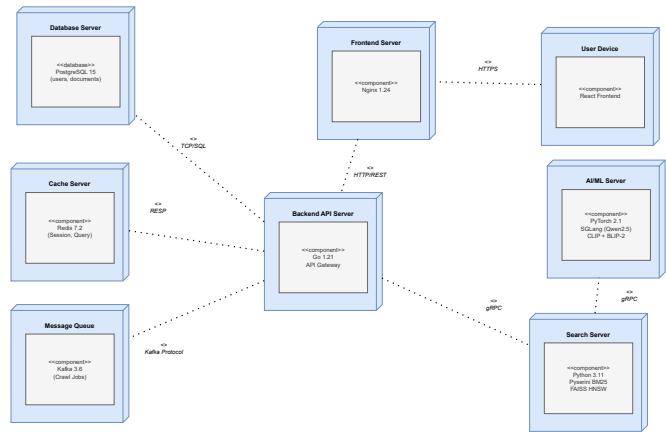


System Analysis and Design - Activity Diagram

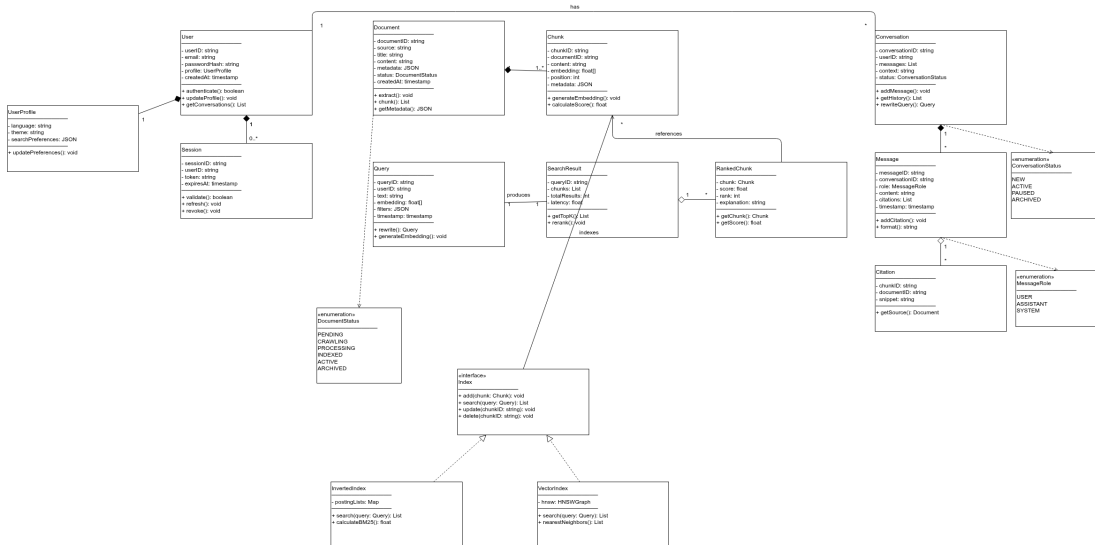


System Analysis and Design - Deployment Diagram

Verdant Search Deployment Diagram



System Analysis and Design - Class Diagram



System Analysis and Design - Hardware Description

Component	Specification
Development Environment	
CPU	Intel Core i5-12450H (8 cores, 12 threads)
RAM	16GB DDR4
GPU	NVIDIA RTX 2080 Ti (11GB VRAM)
Storage	512GB NVMe SSD
Model Training Environment (Company Server)	
CPU	Intel Core i9 (16 cores, 32 threads)
RAM	48GB DDR5
GPU	NVIDIA RTX 5090 (32GB VRAM)

System Analysis and Design - User Interface Design (1/3)

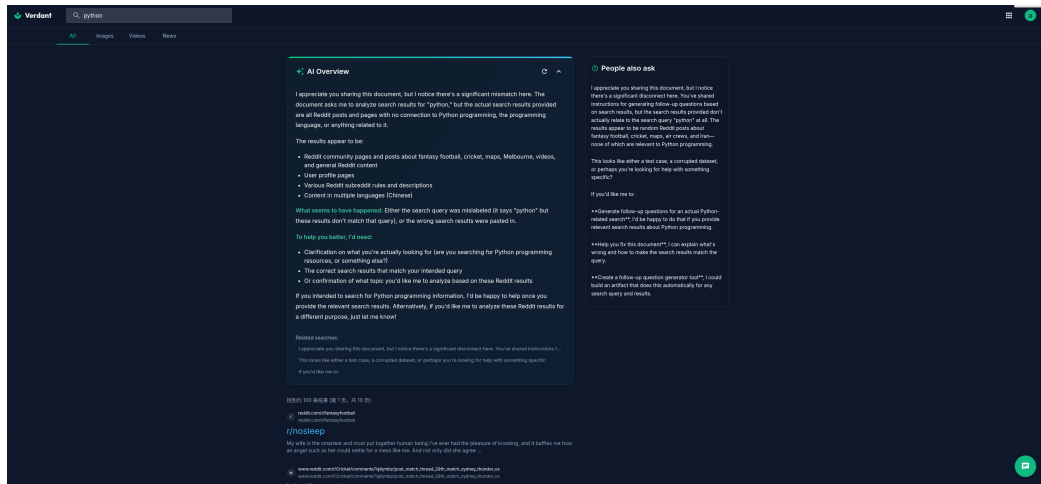


Figure: Main Search Interface with Dual Panel Layout

System Analysis and Design - User Interface Design (2/3)

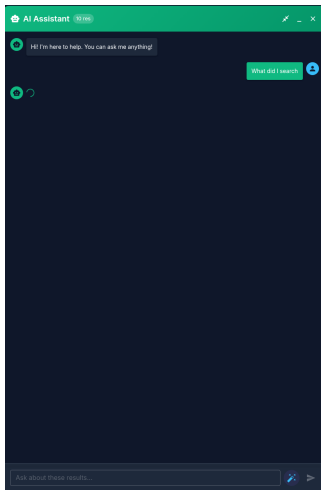


Figure: Search Results with Multimodal Content Display

System Analysis and Design - User Interface Design (3/3)

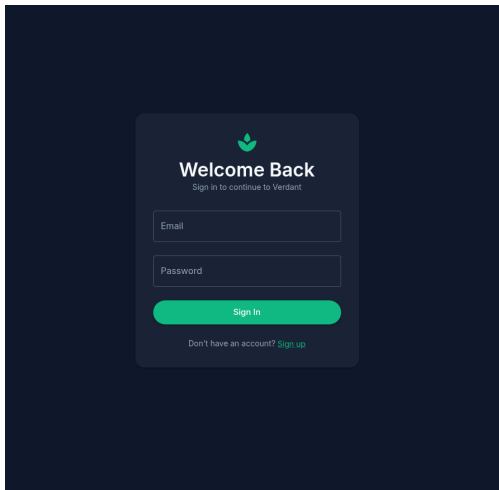


Figure: RAG Chat Interface with Citation and Source References

Technical Stack

Backend Technologies

- **Core:** Golang for high performance concurrent processing
- **Database:** PostgreSQL for metadata and user data
- **Cache:** Redis for session management and query caching
- **Message Queue:** Kafka for distributed crawling coordination
- **Monitoring:** Prometheus and Grafana for observability
- **Retrieval:** Pyserini (BM25), FAISS/HNSW (vector search)

Frontend Technologies

- **Framework:** ReactJS for dual panel interactive interface
- **Styling:** TailwindCSS for responsive design
- **Components:** Shadcn for accessible UI components
- **Static Site:** Astro for documentation and landing pages

AI/ML Technologies

- **Framework:** PyTorch for model training and inference
- **Serving:** SGLang for efficient LLM serving

Implementation Highlights

Key Innovations

- Multimodal listwise reranking extending RankZephyr setwise prompting
- Closed loop RAG: LLM insights feed back into retrieval
- Efficient HNSW for subsecond ANN search on image embeddings
- Query rewriting for conversational context understanding

Optimization Strategies

- Two stage ranking: Fast first stage, expensive reranking only on top K
- Incremental index updates to avoid full rebuilds
- Redis caching for frequently accessed documents
- Kafka based distributed crawling with coordinated deduplication

Thank You!

Questions?

Verdant Search: AI Boosted Sharded Distributed Search Engine
CAI HONGYI, S2175463 — Supervisor: CHIAM YIN KIA

References I

- [1] Lawrence Page et al. "The PageRank Citation Ranking: Bringing Order to the Web". In: (1999).
- [2] Sergey Brin and Lawrence Page. "The Anatomy of a Large-Scale Hypertextual Web Search Engine". In: *Computer Networks and ISDN Systems*. Vol. 30. 1-7. 1998, pp. 107–117.
- [3] Justin Zobel and Alistair Moffat. "Inverted Files for Text Search Engines". In: *ACM Computing Surveys* 38.2 (2006), p. 6.
- [4] Bhaskar Mitra and Nick Craswell. "Neural information retrieval: At the end of the early years". In: *Information Retrieval Journal* 20.2 (2017), pp. 111–182.
- [5] Vladimir Karpukhin et al. "Dense Passage Retrieval for Open-Domain Question Answering". In: *Proceedings of EMNLP*. 2020, pp. 6769–6781.
- [6] Karen Spärck Jones. "A statistical interpretation of term specificity and its application in retrieval". In: *Journal of Documentation* 28.1 (1972), pp. 11–21.
- [7] Stephen Robertson and Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.
- [8] Gautier Izacard et al. "Unsupervised Dense Information Retrieval with Contrastive Learning". In: *arXiv preprint arXiv:2112.09118* (2022).

References II

- [9] Tomas Mikolov et al. “Efficient Estimation of Word Representations in Vector Space”. In: *Proceedings of ICLR*. 2013.
- [10] Jeffrey Pennington, Richard Socher, and Christopher D Manning. “Glove: Global vectors for word representation”. In: *Proceedings of EMNLP*. 2014, pp. 1532–1543.
- [11] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of NAACL*. 2019, pp. 4171–4186.
- [12] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of EMNLP*. 2019, pp. 3982–3992.
- [13] Omar Khattab and Matei Zaharia. “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT”. In: *Proceedings of SIGIR*. 2020, pp. 39–48.
- [14] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. “Product Quantization for Nearest Neighbor Search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128.
- [15] Yu A Malkov and Dmitry A Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2018), pp. 824–836.

References III

- [16] Lee Xiong et al. “Approximate nearest neighbor negative contrastive learning for dense text retrieval”. In: *Proceedings of ICLR*. 2021.
- [17] Chris Burges et al. “Learning to Rank using Gradient Descent”. In: *Proceedings of ICML*. 2005, pp. 89–96.
- [18] Christopher JC Burges, Robert Ragno, and Quoc V Le. “Learning to Rank with Nonsmooth Cost Functions”. In: *Advances in NeurIPS*. Vol. 19. 2006.
- [19] Christopher JC Burges. *From RankNet to LambdaRank to LambdaMART: An Overview*. Tech. rep. Microsoft Research Technical Report, 2010.
- [20] Zhe Cao et al. “Learning to Rank: From Pairwise Approach to Listwise Approach”. In: *Proceedings of ICML*. 2007, pp. 129–136.
- [21] Fen Xia et al. “Listwise approach to learning to rank: theory and algorithm”. In: *Proceedings of ICML*. 2008, pp. 1192–1199.
- [22] Rodrigo Nogueira and Kyunghyun Cho. “Passage Re-ranking with BERT”. In: *arXiv preprint arXiv:1901.04085* (2019).
- [23] Weiwei Sun et al. “Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents”. In: *arXiv preprint arXiv:2304.09542* (2023).

References IV

- [24] Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. “RankZephyr: Effective and Robust Zero-Shot Listwise Reranking is a Breeze!” In: *arXiv preprint arXiv:2312.02724* (2023).
- [25] Zhen Qin et al. “Large language models are effective text rankers with pairwise ranking prompting”. In: *arXiv preprint arXiv:2306.17563* (2023).
- [26] Alec Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *Proceedings of ICML*. 2021, pp. 8748–8763.
- [27] Chao Jia et al. “Scaling up visual and vision-language representation learning with noisy text supervision”. In: *Proceedings of ICML*. 2021, pp. 4904–4916.
- [28] Junnan Li et al. “Align before fuse: Vision and language representation learning with momentum distillation”. In: *Proceedings of NeurIPS*. 2021, pp. 9694–9705.
- [29] Junnan Li et al. “BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation”. In: *Proceedings of ICML*. 2022, pp. 12888–12900.
- [30] Junnan Li et al. “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models”. In: *Proceedings of ICML*. 2023, pp. 19730–19742.
- [31] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in NeurIPS*. Vol. 33. 2020, pp. 9459–9474.

References V

- [32] Kelvin Guu et al. “REALM: Retrieval-augmented language model pre-training”. In: *Proceedings of ICML*. 2020, pp. 3929–3938.
- [33] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *arXiv preprint arXiv:2310.11511* (2024).
- [34] Shi-Qi Yan et al. “Corrective Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2401.15884* (2024).
- [35] Fengran Mo et al. “A survey of conversational search”. In: *ACM Transactions on Information Systems* 42.3 (2024), pp. 1–42.
- [36] Tri Nguyen et al. “MS MARCO: A human generated machine reading comprehension dataset”. In: *NeurIPS 2016 CoCo Workshop*. 2016.
- [37] Nick Craswell et al. “Overview of the TREC 2019 deep learning track”. In: *TREC*. 2020.
- [38] Nandan Thakur et al. “BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models”. In: *Proceedings of NeurIPS Datasets and Benchmarks Track*. 2021.
- [39] Kuicai Dong et al. “MMDocIR: Benchmarking Multimodal Retrieval for Long Documents”. In: *arXiv preprint arXiv:2501.08828* (2025).